

Complete Data Cleaning Notes (Beginner to Advanced)

What is Data Cleaning?

Data Cleaning is the process of identifying and correcting or removing errors, inconsistencies, duplicate records, missing values, incorrect data types, and outliers to improve data quality before analysis or machine learning.

Example

Raw Data

ID	Name	Age	Salary
1	John	22	25000
2	John	22	25000
3	NaN	30	50000
4	Mike	-5	1000000

Clean Data

ID	Name	Age	Salary
1	John	22	25000
3	Unknown	30	50000

Data Cleaning Workflow

Import Dataset



Understand Dataset



Check Missing Values



Handle Missing Values



Remove Duplicates



Fix Data Types



Clean Text Data



Handle Outliers



Feature Scaling



Encoding



Final Clean Dataset

Step 1: Import Libraries

```
import pandas as pd
```

```
import numpy as np
```

Step 2: Load Dataset

```
df = pd.read_csv("data.csv")
```

Other formats:

```
df = pd.read_excel("data.xlsx")
```

```
df = pd.read_json("data.json")
```

Step 3: Explore the Dataset

First 5 Rows

`df.head()`

Last 5 Rows

`df.tail()`

Random Rows

`df.sample(5)`

Shape

`df.shape`

Step 4: Dataset Information

`df.info()`

Step 5: Summary Statistics

`df.describe()`

`df.describe(include="all")`

Step 6: Check Missing Values

Find Missing Values

`df.isnull()`

Count Missing Values

`df.isnull().sum()`

Percentage

`(df.isnull().sum()/len(df))*100`

Step 7: Handle Missing Values

Remove Rows

`df = df.dropna()`

Remove Columns

`df = df.dropna(axis=1)`

Fill Numeric Values

Mean

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

Median

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Mode

```
df["Age"] = df["Age"].fillna(df["Age"].mode()[0])
```

Fill Categorical Values

```
df["City"] = df["City"].fillna("Unknown")
```

Step 8: Duplicate Records

Check

```
df.duplicated()
```

Count

```
df.duplicated().sum()
```

Remove

```
df = df.drop_duplicates()
```

Step 9: Rename Columns

```
df = df.rename(columns={  
    "Age": "age",  
    "Salary": "salary"  
})
```

Step 10: Data Types

```
df.dtypes
```

```
df["Age"] = df["Age"].astype(int)
```

```
df["Salary"] = df["Salary"].astype(float)
```

```
df["Name"] = df["Name"].astype(str)
```

Step 11: String Cleaning

```
df["Name"] = df["Name"].str.strip()
df["City"] = df["City"].str.lower()
df["City"] = df["City"].str.upper()
df["City"] = df["City"].str.title()
df["Gender"] = df["Gender"].replace({
    "M": "Male",
    "F": "Female"
})
```

Step 12: Unique Values

```
df["City"].unique()
df["City"].nunique()
df["City"].value_counts()
```

Step 13: Remove Special Characters

```
df["Salary"] = df["Salary"].str.replace("$", "")
df["Salary"] = df["Salary"].str.replace(",", "")
df["Salary"] = df["Salary"].astype(float)
```

Step 14: Date Cleaning

```
df["Date"] = pd.to_datetime(df["Date"])
df["Year"] = df["Date"].dt.Year
df["Month"] = df["Date"].dt.month
df["Day"] = df["Date"].dt.day
df["Weekday"] = df["Date"].dt.day_name()
```

Step 15: Outlier Detection

What is an Outlier?

An outlier is a value that is significantly higher or lower than the rest of the data.

12 15 18 20 22 25 400

400 is an outlier.

IQR Method

Formula

$Q1 = 25\text{th Percentile}$

$Q3 = 75\text{th Percentile}$

$IQR = Q3 - Q1$

Lower Bound

$Q1 - 1.5 * IQR$

Upper Bound

$Q3 + 1.5 * IQR$

Detect Outliers

```
Q1 = df["salary"].quantile(0.25)
```

```
Q3 = df["salary"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower = Q1 - 1.5 * IQR
```

```
upper = Q3 + 1.5 * IQR
```

```
outliers = df[
```

```
    (df["salary"] < lower) |
```

```
    (df["salary"] > upper)
```

```
]
```

```
print(outliers)
```

Remove Outliers

```
df = df[
```

```
(df["salary"] >= lower) &
(df["salary"] <= upper)
]
```

Multiple Numeric Columns

```
numeric_columns = df.select_dtypes(include="number").columns
```

```
for col in numeric_columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df = df[
        (df[col] >= lower) &
        (df[col] <= upper)
    ]
```

Step 16: Z-Score Method

```
from scipy.stats import zscore
z = np.abs(zscore(df["salary"]))
```

```
df = df[z < 3]
```

Step 17: Remove Unnecessary Columns

```
df = df.drop(columns=["ID"])
```

Step 18: Sorting

Ascending

```
df.sort_values("Salary")
```

Descending

```
df.sort_values("Salary", ascending=False)
```

Step 19: Reset Index

```
df = df.reset_index(drop=True)
```

Step 20: Feature Scaling

Min-Max Scaling

Formula

$$X' = (X - \text{Min}) / (\text{Max} - \text{Min})$$

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
df[["Salary"]] = scaler.fit_transform(df[["Salary"]])
```

Standard Scaling

Formula

$$Z = (X - \text{Mean}) / \text{Standard Deviation}$$

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
df[["Salary"]] = scaler.fit_transform(df[["Salary"]])
```

Step 21: Mapping

What is Mapping?

Mapping is the process of replacing existing values in a column with new values based on a predefined dictionary.

It is commonly used to:

Convert categorical values into numerical values

Standardize inconsistent values

Rename categories

Prepare data for machine learning

```
df["Gender"] = df["Gender"].map({  
    "Male": 1,  
    "Female": 0  
})
```

Step 22: Final Dataset Validation

Complete Data Cleaning Pipeline

```
import pandas as pd
```

```
# Load dataset
```

```
df = pd.read_csv("data.csv")
```

```
# Remove duplicate rows
```

```
df = df.drop_duplicates()
```

```
# Fill missing values
```

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

```
df["Salary"] = df["Salary"].fillna(df["Salary"].median())
```

```
# Clean strings
```

```
df["Name"] = df["Name"].str.strip()
```

```
df["City"] = df["City"].str.title()
```

```
# Remove outliers
```

```
Q1 = df["Salary"].quantile(0.25)
```

```
Q3 = df["Salary"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower = Q1 - 1.5 * IQR
```

```
upper = Q3 + 1.5 * IQR
```

```
df = df[  
    (df["Salary"] >= lower) &  
    (df["Salary"] <= upper)  
]
```

```
# Reset index
```

```
df = df.reset_index(drop=True)
```

```
# Verify dataset
```

```
print(df.info())
```

```
print(df.head())
```

Best Practices

Always inspect the dataset before cleaning.

Keep a backup of the original dataset.

Understand why data is missing before filling it.

Remove duplicates before training.

Verify data types.

Handle outliers carefully (don't remove them blindly).

Standardize text values.

Scale numerical features when required.

Encode categorical variables before model training.

Validate the cleaned dataset before saving.

Interview Questions

What is data cleaning?

Why is data cleaning important in machine learning?

What are missing values?

What is the difference between `dropna()` and `fillna()`?

When should you use mean, median, or mode?

How do you detect duplicate records?

What are outliers?

Explain the IQR method.

Explain the Z-score method.

What is feature scaling?

What is the difference between Min-Max Scaling and Standard Scaling?

What is Label Encoding?

What is One-Hot Encoding?

Why do we reset the index after removing rows?

What checks should you perform after completing data cleaning?

This covers the complete data cleaning workflow commonly used in data analysis and machine learning projects.